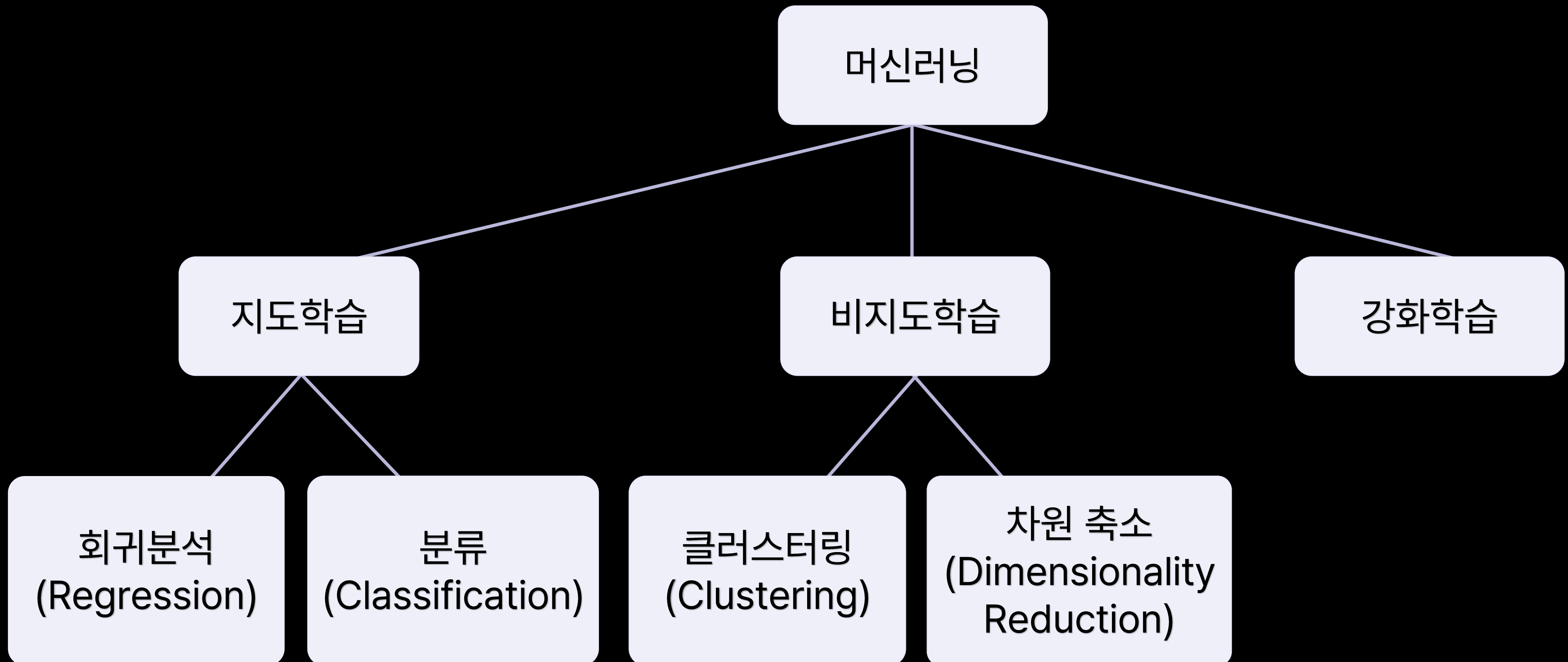

머신러닝 II

02 분류

목차 02 분류

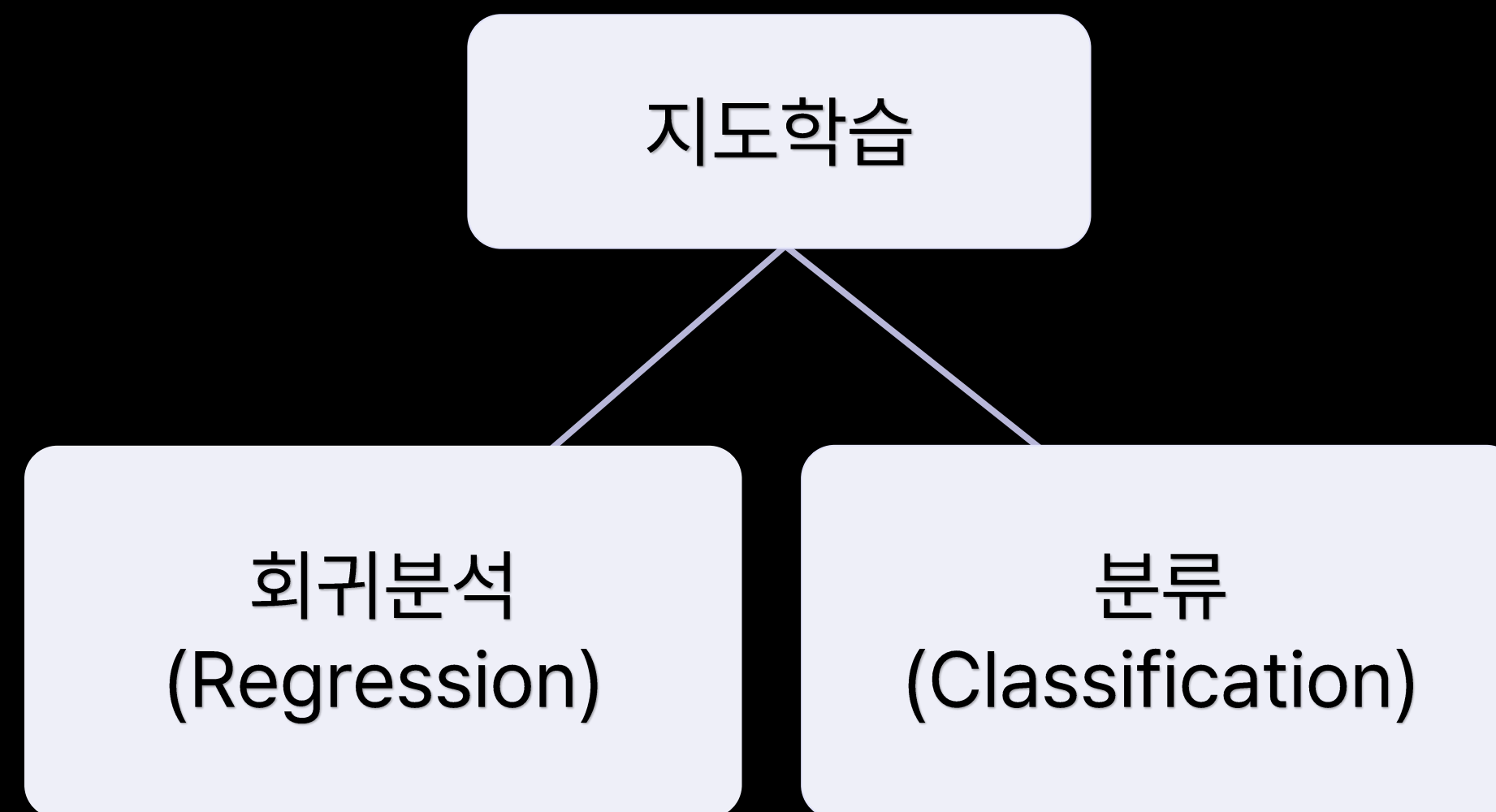
1. 분류 개념과 로지스틱 회귀
2. Support Vector Machine(SVM)
3. 나이브 베이즈 분류
4. KNN (K-Nearest Neighbor)
5. 분류 알고리즘 평가 지표

1. 분류 개념과 로지스틱 회귀

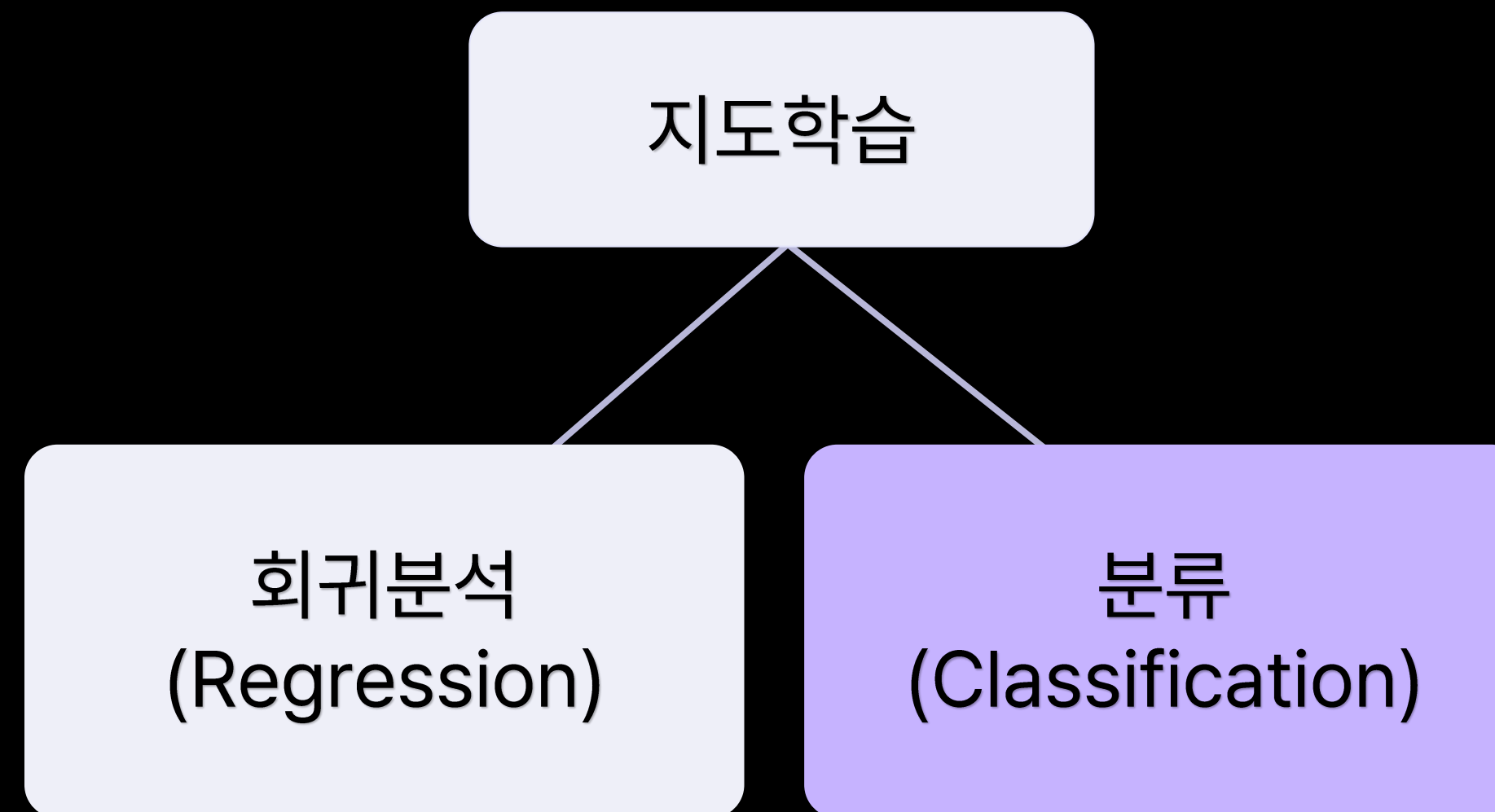


컴퓨터에 **정답이 있는** 데이터를 학습시키는 방법

- 주어진 입력으로부터 출력 값을 예측하고자 할 때 사용



주어진 입력 값이 어떤 클래스에 속할지에 대한 결과 값을 도출하는 알고리즘



- 해외 여행을 준비하고 있다고 가정하기
- 완벽한 여행을 위해 항공 지연을 피하고자 한다.
- 이 때, 기상 정보(구름 양, 풍속)를 활용하여 해당 항공의 지연 여부를 예측할 수 있다면?





문제 정의와 해결 방안

머신러닝 II

02 분류

1. 분류 개념과 로지스틱 회귀

X

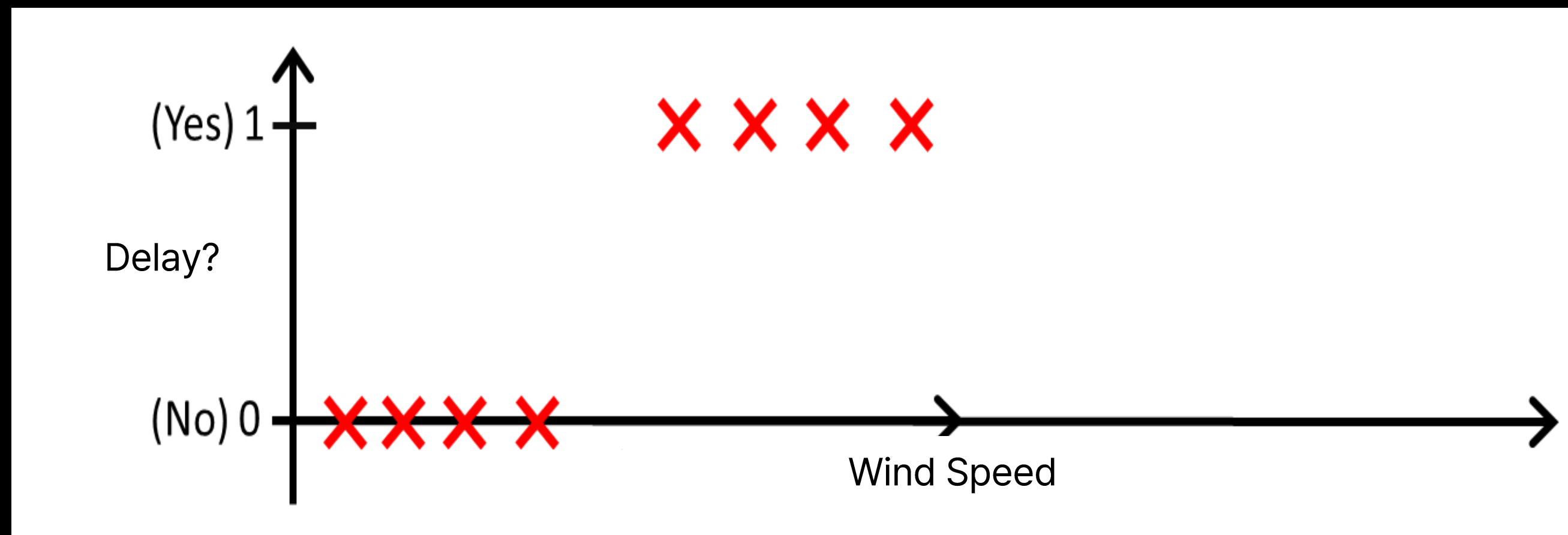
Y

- 데이터 : 과거 기상 정보(풍속)와 그에 따른 항공 지연 여부
- 목표 : 기상 정보에 따른 항공 지연 여부 예측하기
- 해결 방안 : 분류 알고리즘

X	Y
풍속(m/s)	지연 여부
2	NO
4	YES
3	NO
1	NO

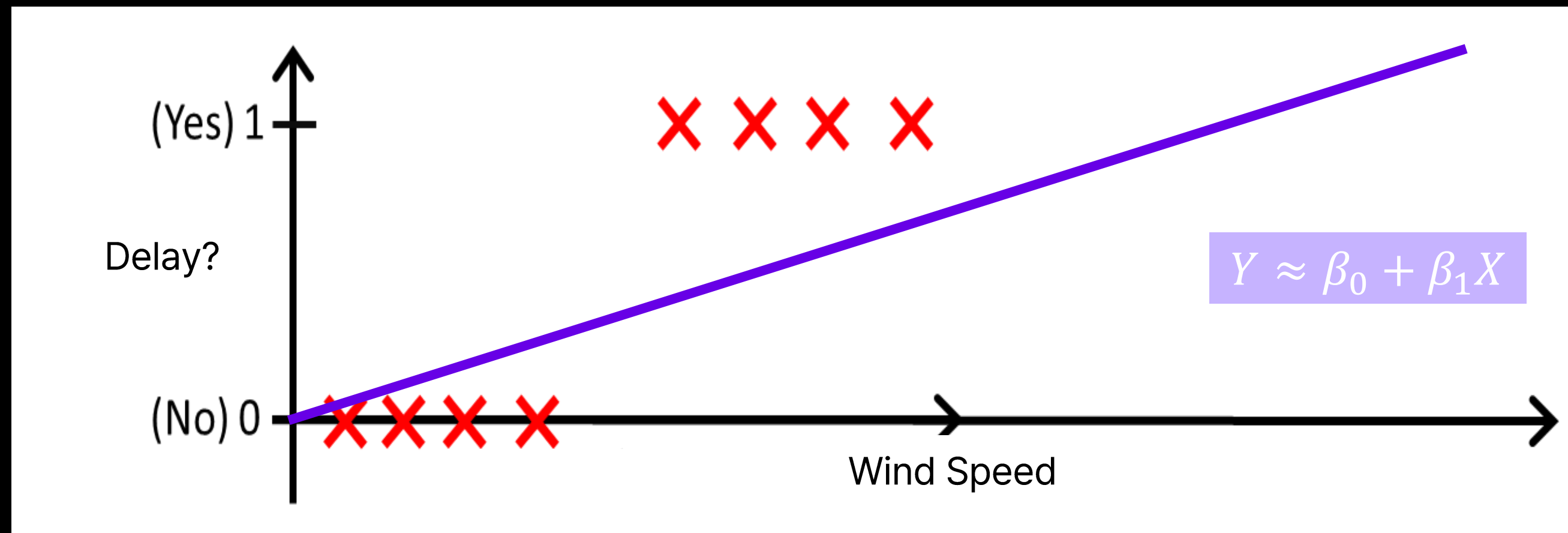


- 다양한 분류 알고리즘이 존재하며, **예측 목표와 데이터 유형에 따라** 적용



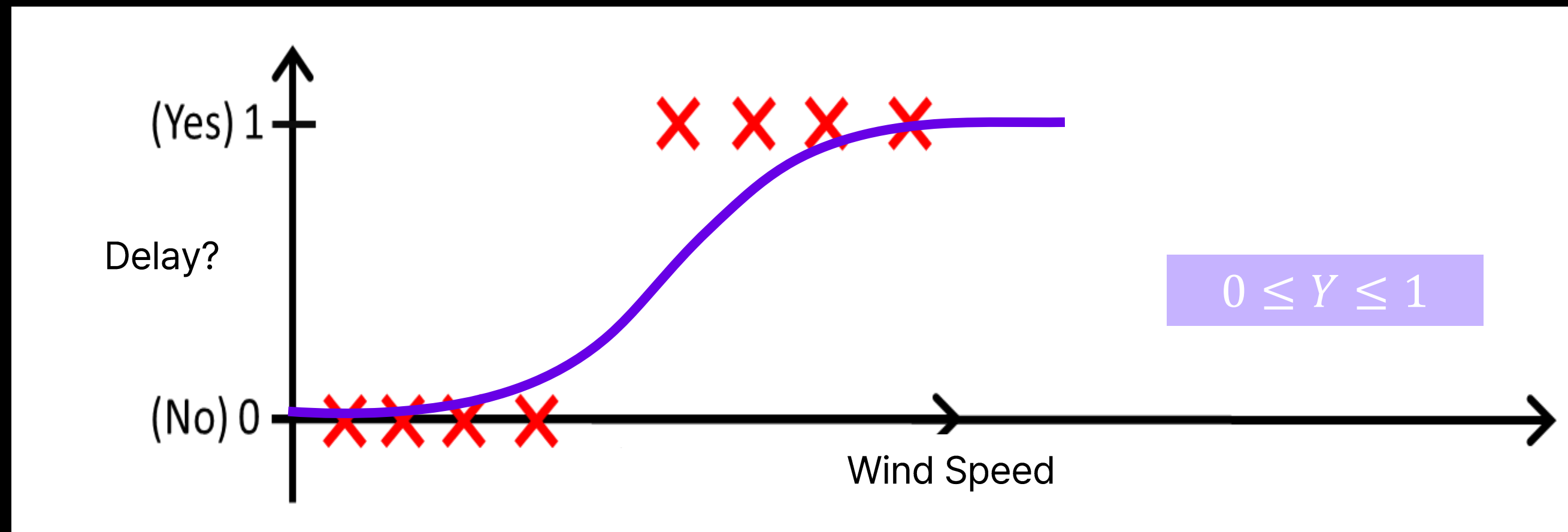


- 일반적인 회귀 알고리즘은 분류 문제에 그대로 사용할 수 없음
 - 선형 회귀는 $-\infty \sim +\infty$ 의 값을 가질 수 있음
- 우리의 목표는 지연 여부 판별인데, 결과 값이 1000이라면?



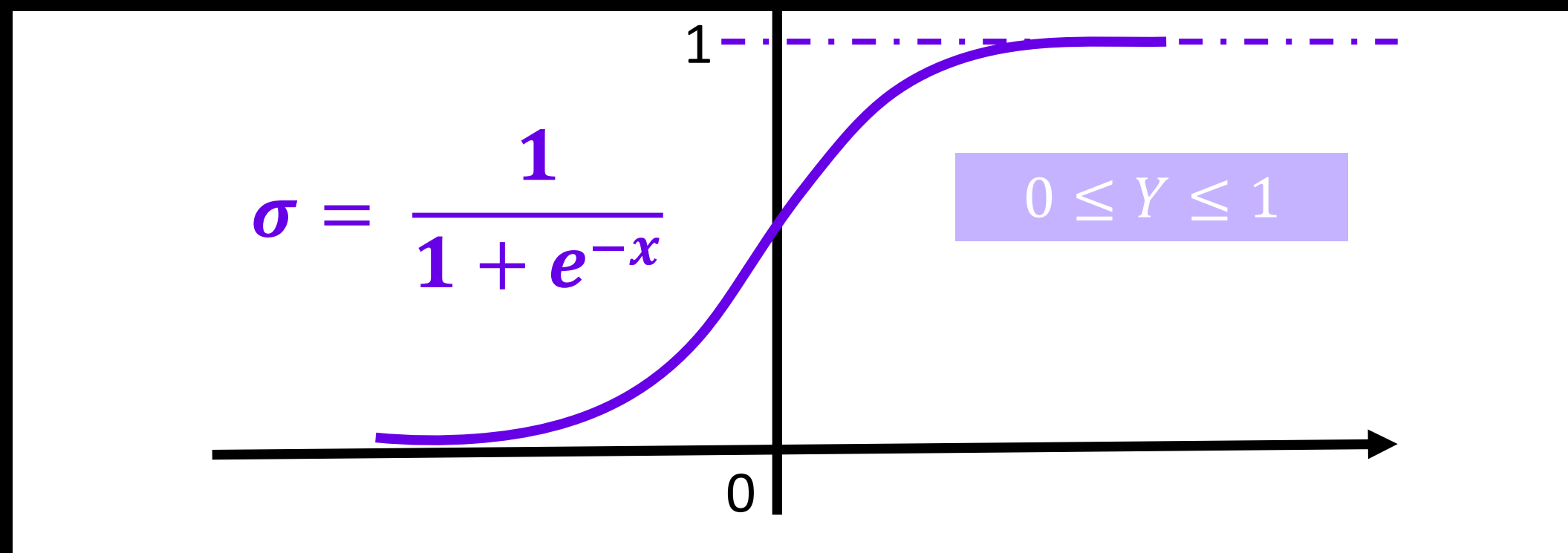


- 해당 클래스에 속할 확률인 0 또는 1 사이의 값만 내보낼 수 있도록 선형 회귀 알고리즘 수정
 - 분류 문제에 적용하기 위해 출력 값의 범위를 수정한 회귀
 - 로지스틱 회귀(Logistic Regression)





이진 분류(Binary Classification) 문제를 해결하기 위한 모델



최소값 0, 최대값 1로 결과값을 수렴시키기 위해 **Sigmoid(logistic) 함수** 사용



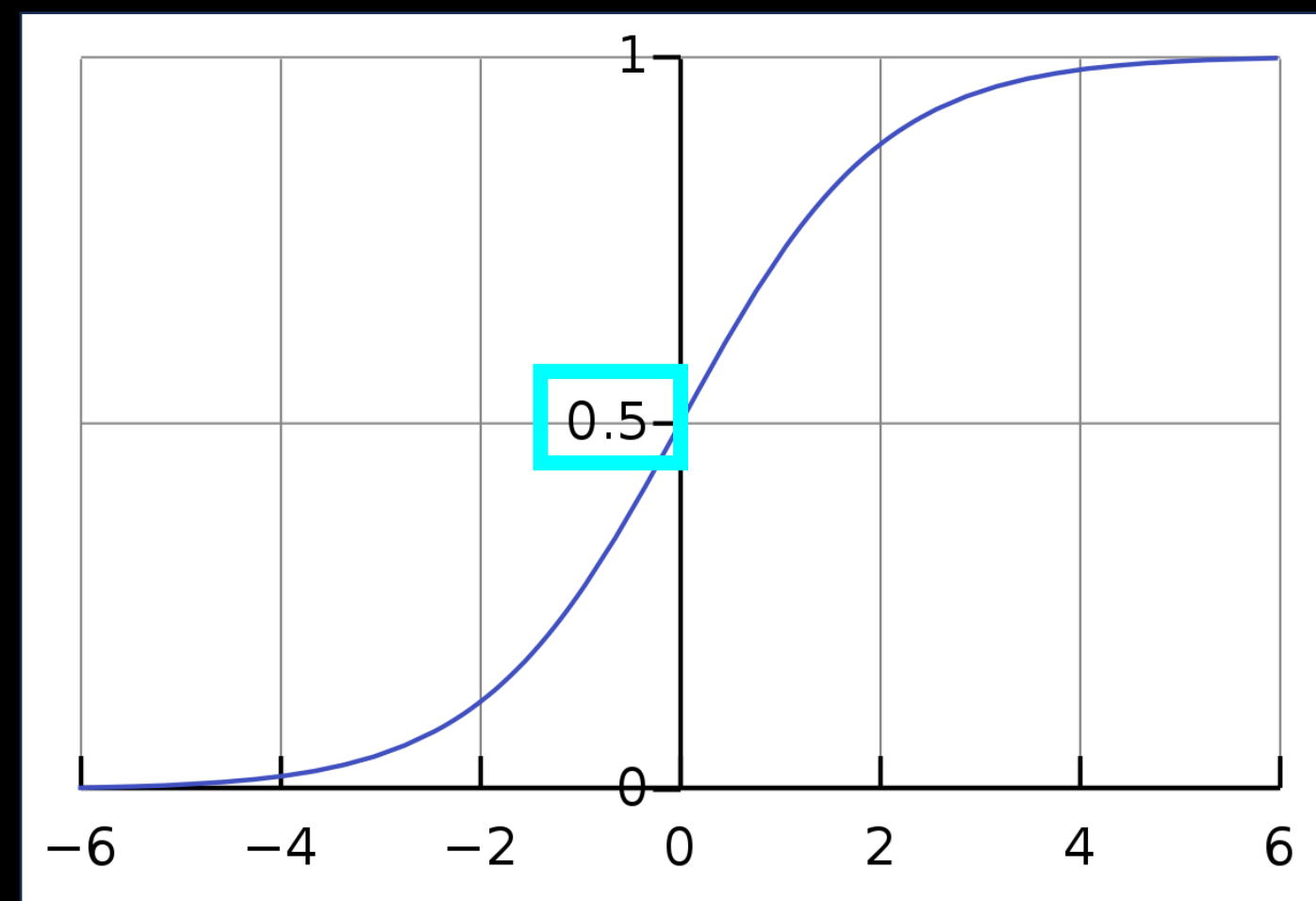
$$g(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}$$

- **s자형 곡선**을 갖는 함수
- x 값이 커질 경우, $g(x)$ 값은 점점 1에 수렴
- x 값이 작아질 경우, $g(x)$ 값은 점점 0에 수렴



데이터를 분류하는 기준 값

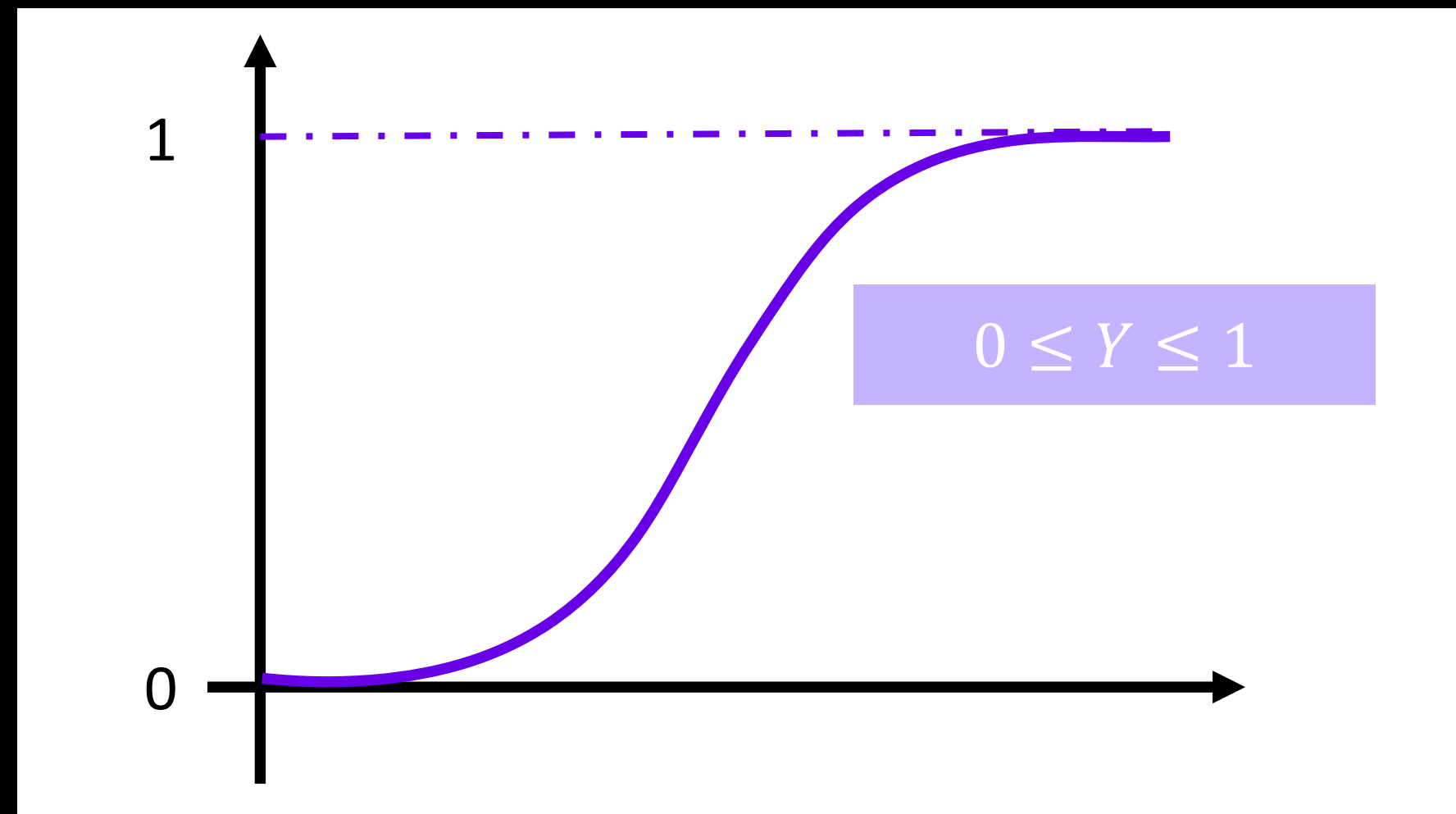
- 일반적으로 **출력 값(확률) 0.5**를 기준으로 판별



Sigmoid(logistic) function



- 주로 2개 값 분류(이진 분류)를 위해 사용
- 선형 회귀를 응용한 알고리즘이기 때문에 회귀의 특징을 가짐





```
import pandas as pd
from sklearn.model_selection import train_test_split

# 데이터셋 불러오기
df = pd.read_csv('data.csv')

train, test = train_test_split(df, test_size=0.2, random_state=21)
```

- **train_test_split()**

- 데이터 세트를 분리해주는 함수
- *arrays : 데이터셋
- test_size : 테스트 세트의 비율 (0~1)
- random_state : 분할 과정에서의 무작위성 제어



```
from sklearn.linear_model import LogisticRegression
```

데이터 분리

```
train_X, test_X, train_y, test_y = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

```
logistic_model = LogisticRegression()
```

 모델 생성

```
logistic_model.fit(train_X, train_y)
```

 모델 학습

```
predicted = logistic_model.predict(test_X)
```

 테스트 데이터에 대한 예측

- 회귀와 마찬가지로 **데이터 분리 과정이 선행** 되어야 함
- 모델 생성과 학습, 예측하는 과정은 모두 회귀와 동일

2. Support Vector Machine

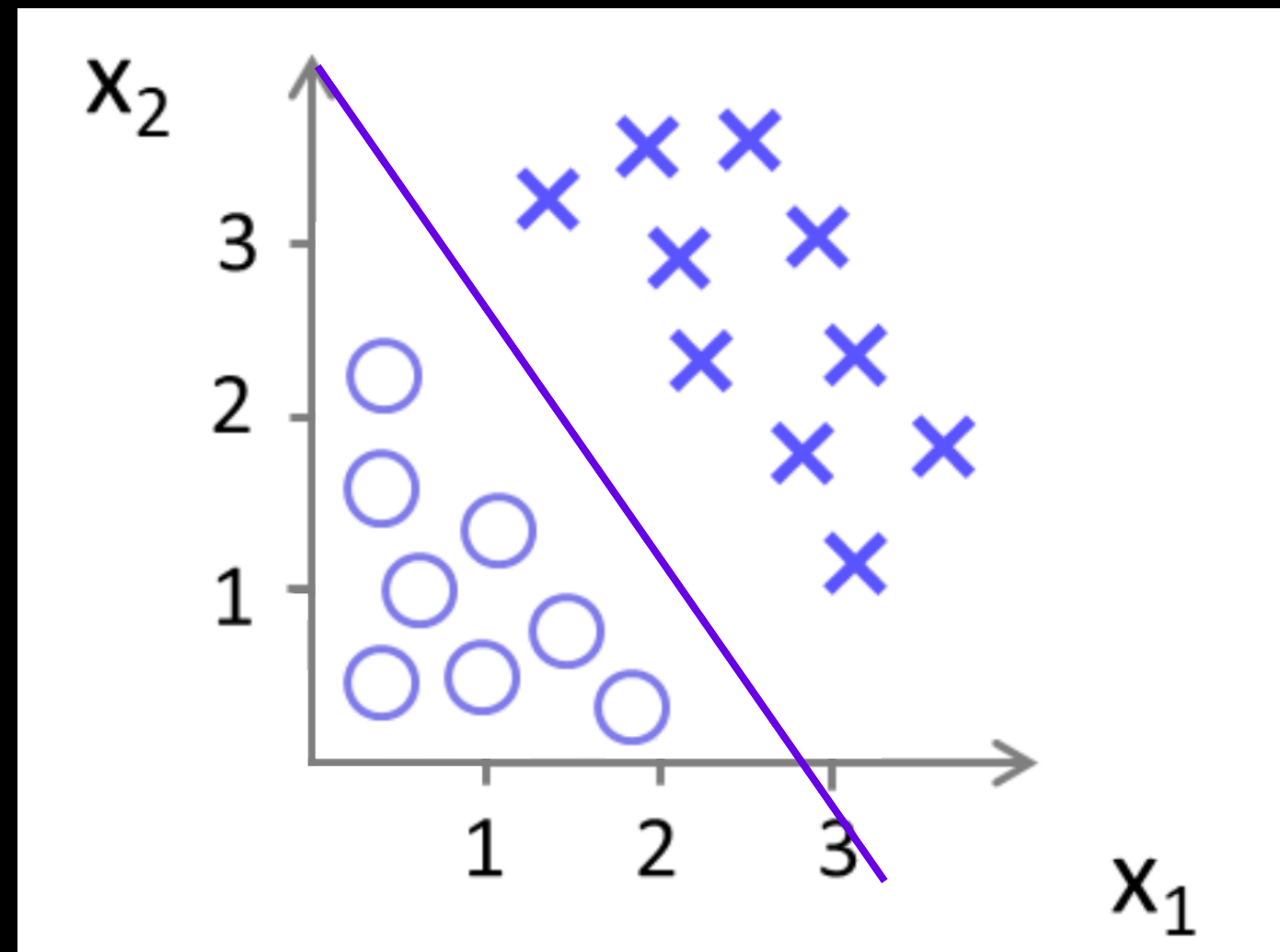


- 문제 정의
 - 양성(1)과 음성(0) 두 개의 결과값으로 분류되는 이진 분류 문제
 - 예) 지연 여부 판별, 이상 거래 판별
- 해결 방안
 - **SVM(Support Vector Machine) 분류 알고리즘**



데이터 포인트들 사이 **최적의 결정 경계**를 정의하는 모델

- 딥러닝 기술 등장 이전까지 가장 인기 있었던 분류 알고리즘

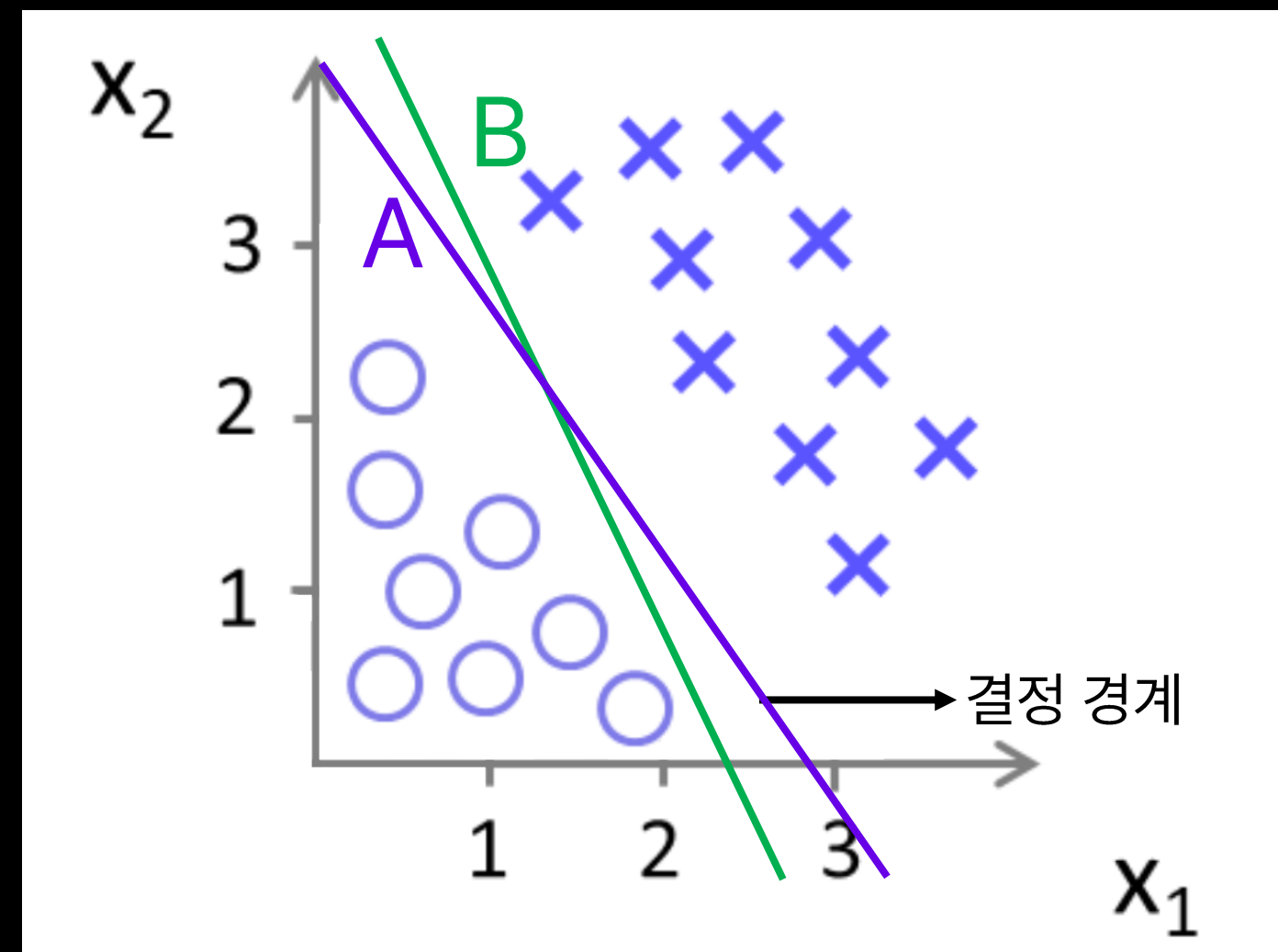




데이터 군으로부터 최대한 멀리 떨어진 경계

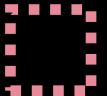
- A와 B 중 더 최적의 결정 경계는?

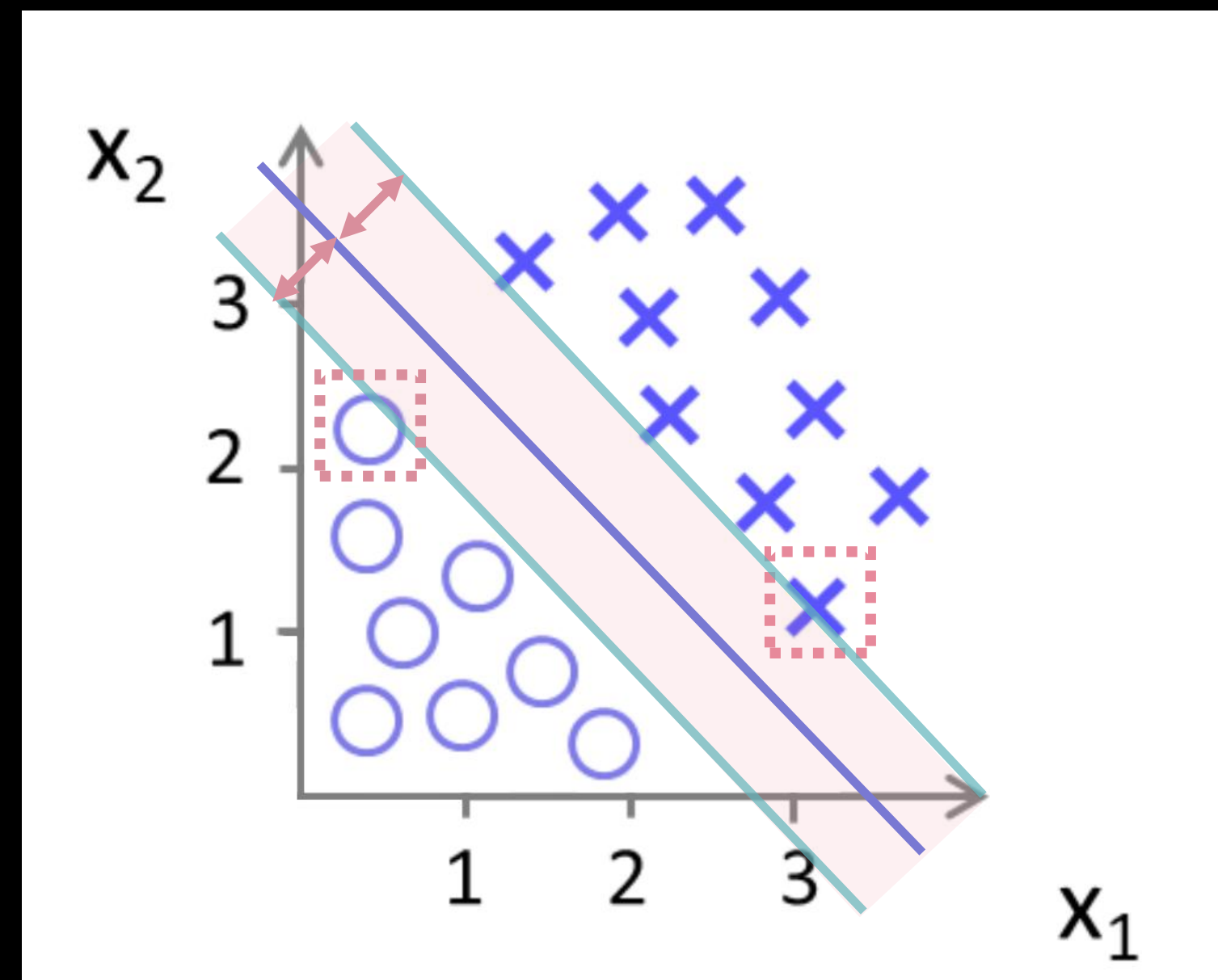
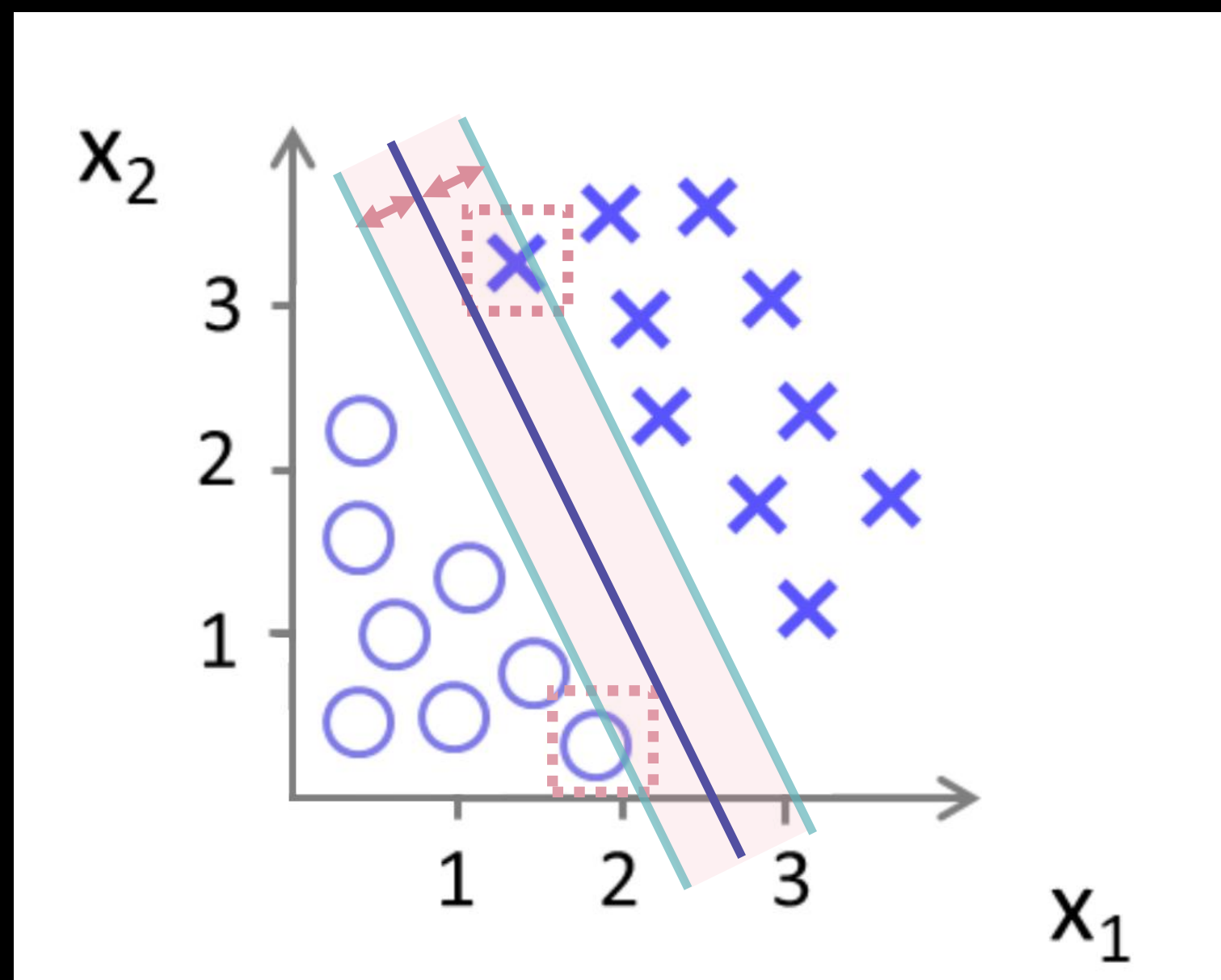
→ 데이터로부터 더 멀리 떨어진 A !





결정 경계와 가장 가까이 있는 데이터 포인트들

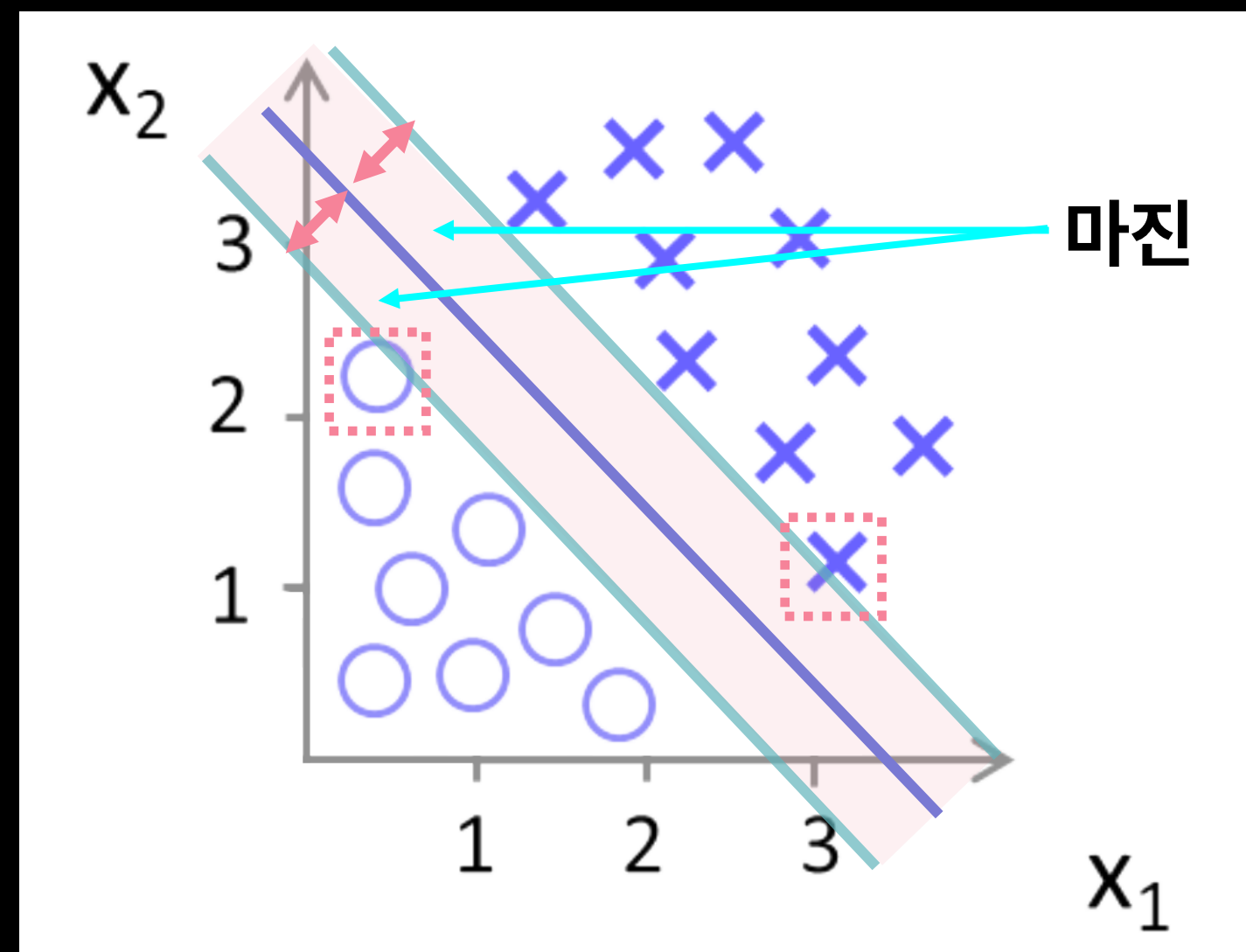
 Support Vector





결정 경계와 서포트 벡터 사이의 거리

- 마진(Margin)이라고 함
- 마진을 최대화 하는 결정 경계를 찾음





Hard Margin vs. Soft Margin

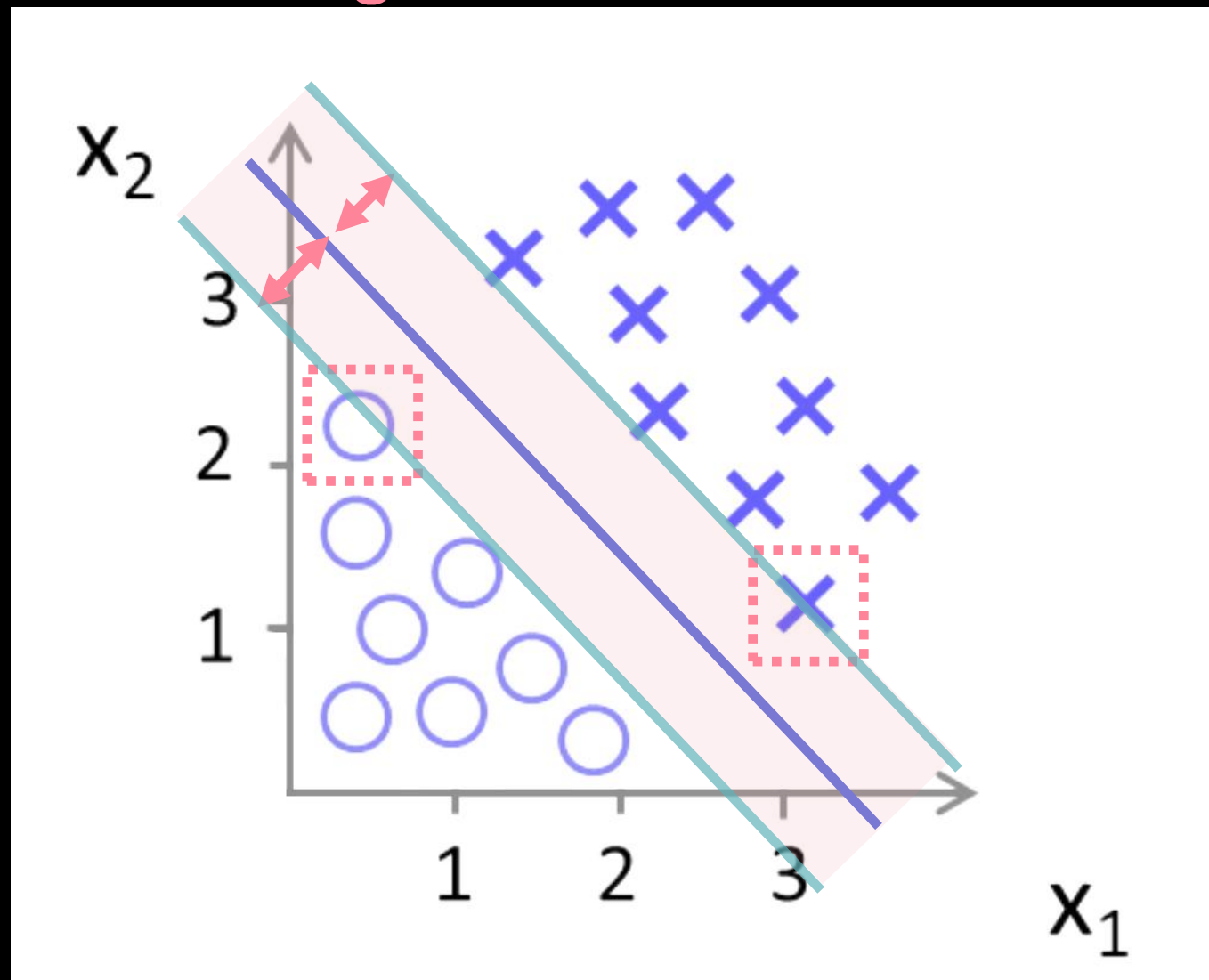
머신러닝 II

02 분류

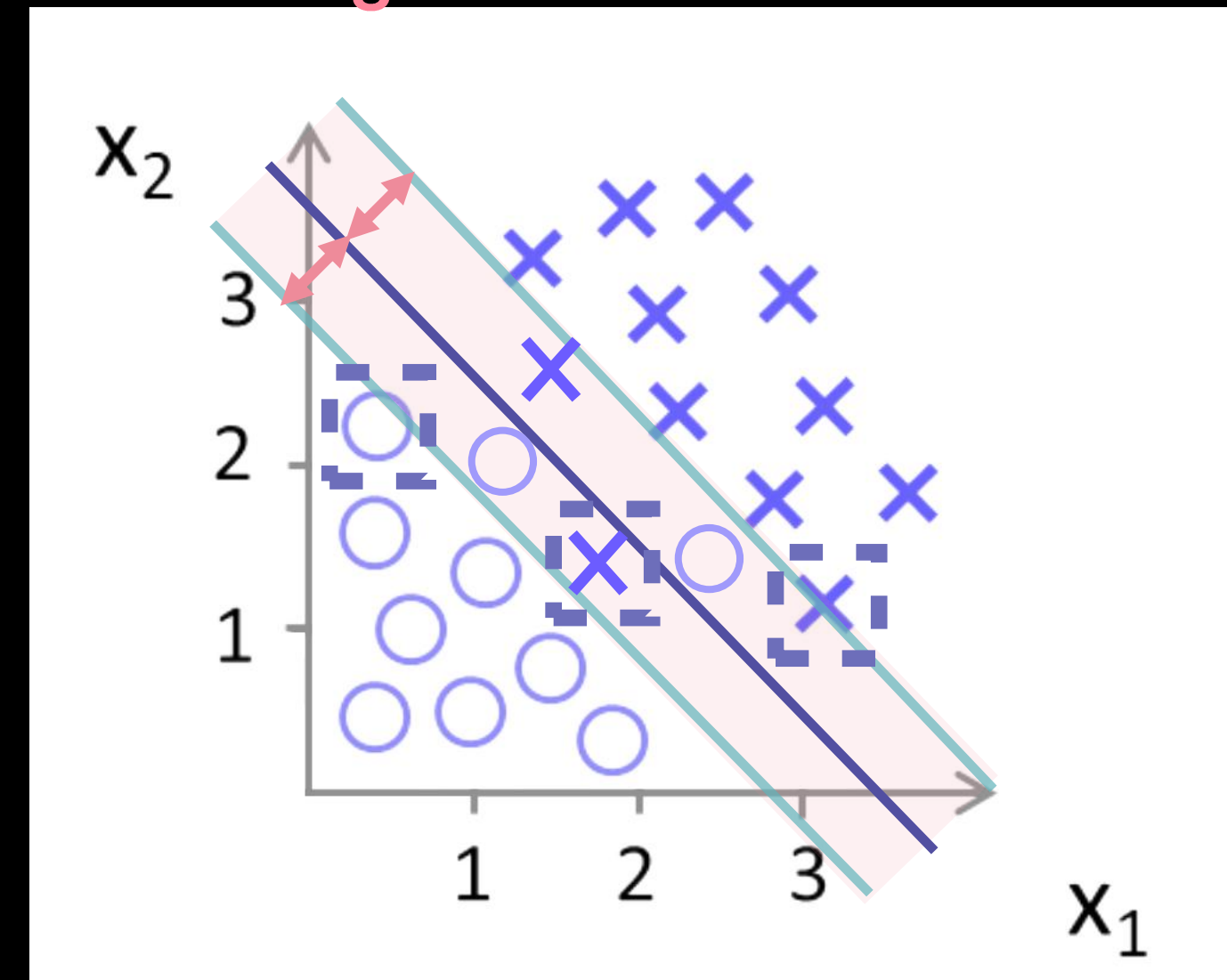
2. Support Vector Machine

- 이상치(Outlier) 허용 범위에 따라 Hard Margin과 Soft Margin으로 구분됨
- Hard Margin : 엄격하게 두 개의 클래스를 분리하는 결정 경계를 구함
- Soft Margin : 서포트 벡터가 위치한 경계선에 약간의 여유 변수를 둘 수 있음

Hard Margin

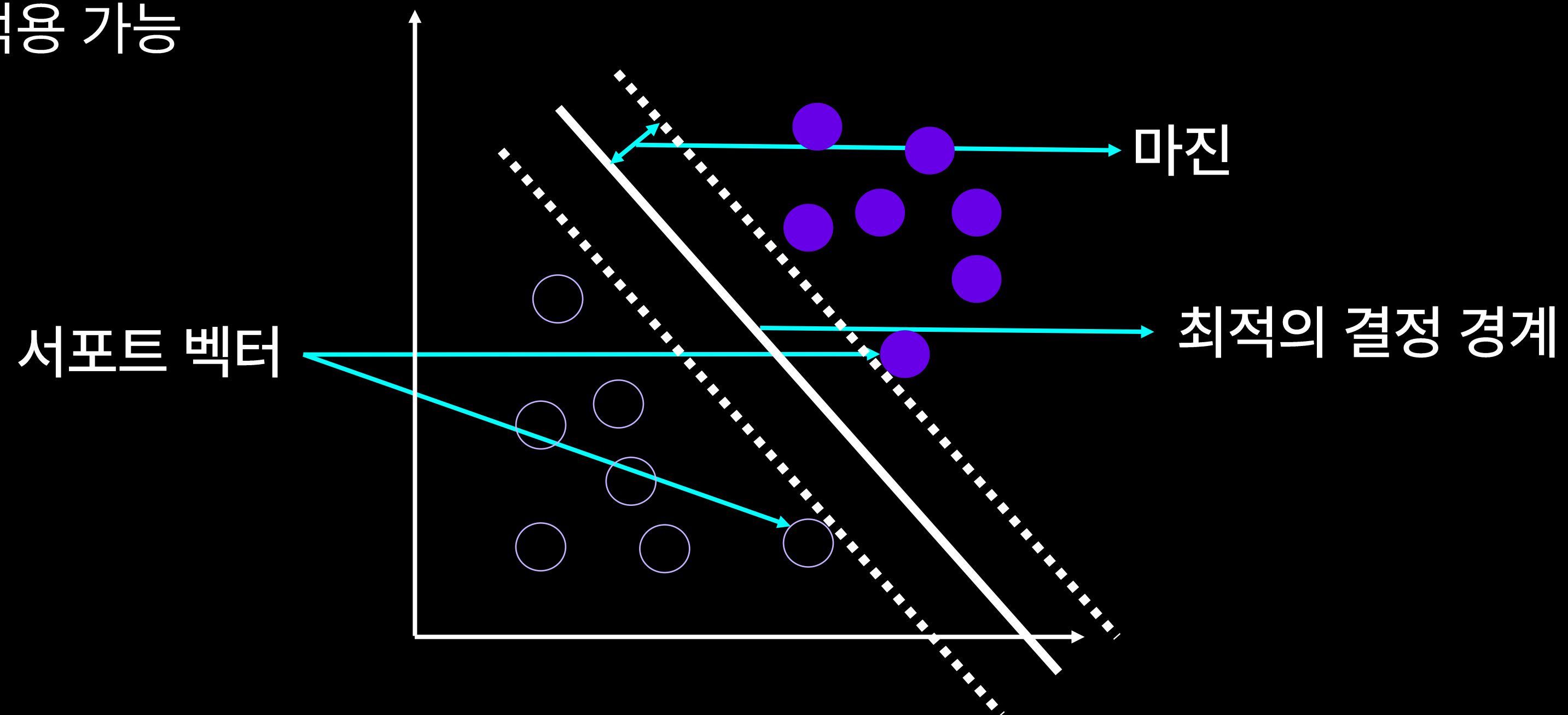


Soft Margin





- 선형 분류와 비선형 분류 모두 가능
- 고차원 데이터에서도 높은 성능의 결과를 도출
- 회귀에도 적용 가능





```
from sklearn.svm import SVC
```

데이터 분리

```
train_X, test_X, train_y, test_y = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

```
svm = SVC()
```

 모델 생성

```
svm.fit(train_X, train_y)
```

 모델 학습

```
predicted = svm.predict(test_X)
```

 테스트 데이터에 대한 예측

- SVM 중 분류 모델을 사용하고 싶을 땐, SVC 를 import

3. 나이브 베이즈 분류



- 문제 정의
 - 10만개의 메일 중 스팸 메일과 정상 메일을 분류하고 싶다면?
- 해결 방안
 - **나이브 베이즈 분류 알고리즘**





각 특징들이 독립적, 서로 영향을 미치지 않을 것이라는 가정 설정

베이즈 정리를 활용한 확률 통계학적 분류 알고리즘

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(B|A)P(A)}{P(B)}$$

베이즈 정리를 활용하여 계산한 조건부 확률



- 맑은 날 비가 오지 않을 확률은?
- $P(\text{비가 안옴} \mid \text{맑은 날})$
$$= \frac{P(\text{맑은 날} \mid \text{비가 안옴}) * P(\text{비가 안옴})}{P(\text{맑은 날})}$$

	비가 옴	비가 안옴	
맑은 날	2	8	10
흐린 날	5	5	10
	7	13	20

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(B|A)P(A)}{P(B)}$$



1. 스팸 메일과 정상 메일의 단어를 체크
2. 새로운 메일의 단어들에 대한 확률로 스팸 메일을 구분
3. $P(\text{스팸} \mid \text{단어1, 단어2, ...}) > P(\text{정상} \mid \text{단어1, 단어2, ...})$ 이면 스팸





- 각 특징들이 **독립이라면** 다른 분류 방식에 비해 **결과가 좋고, 학습 데이터도 적게 필요**
- 각 특징들이 **독립이 아니라면** 즉, 특징들이 서로 영향을 미치면 분류 결과 **신뢰성 하락**
- **학습 데이터에 없는 범주의 데이터일 경우 정상적 예측 불가능**



```
from sklearn.naive_bayes import GaussianNB
```

데이터 분리

```
train_X, test_X, train_y, test_y = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

```
naive_model = GaussianNB() 모델 생성
```

```
naive_model.fit(train_X, train_y) 모델 학습
```

```
predicted = naive_model.predict(test_X) 테스트 데이터에 대한 예측
```

- GaussianNB()
 - 각 특징이 정규분포(가우시안 분포)를 따른다고 가정하는 분류기

4. KNN(K-Nearest Neighbor)



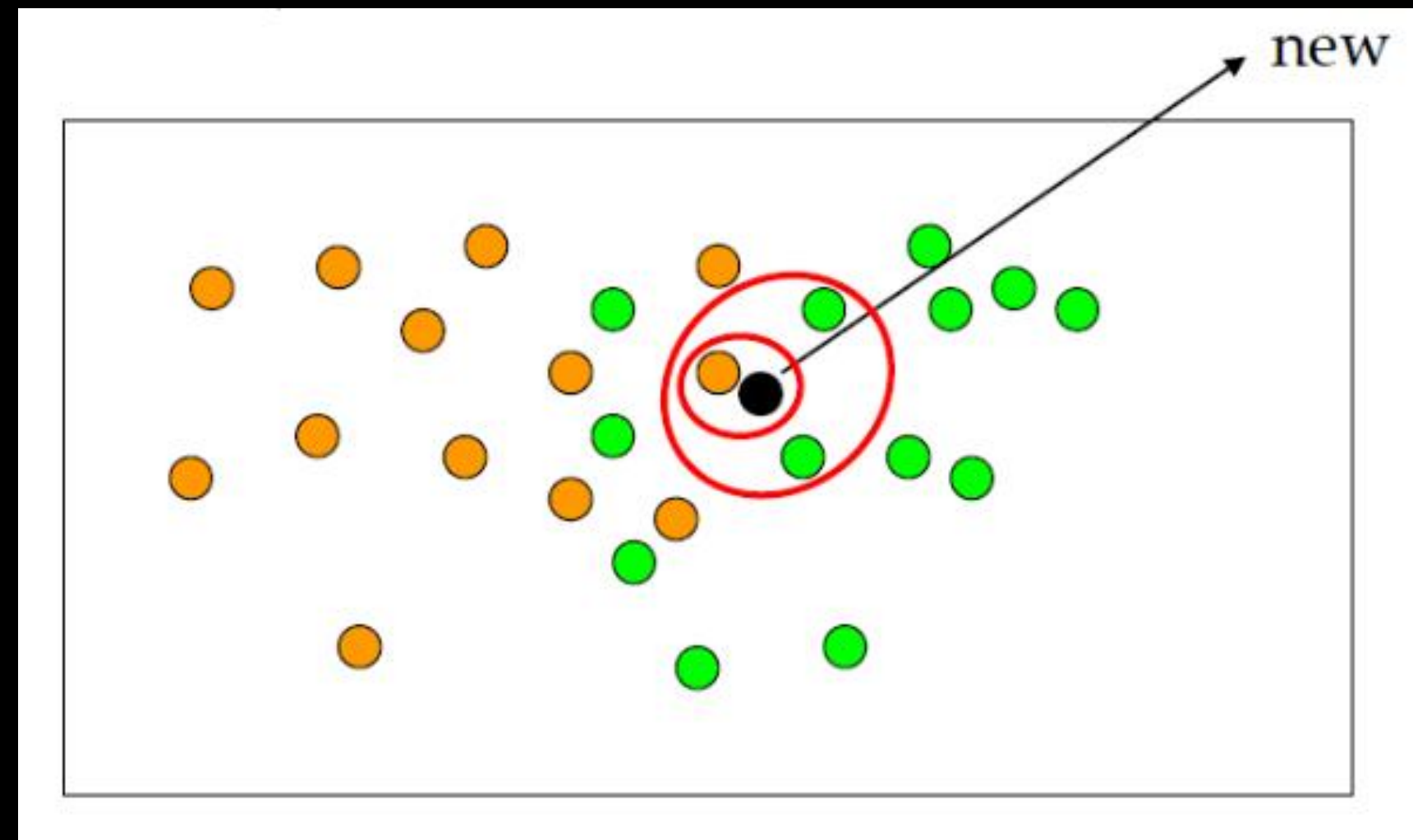
- 문제 정의
 - 고객이 평가한 영화 평점 데이터를 기준으로 기존 보유 고객을 분류한 후 새로 유입된 고객을 기준에 따라 분류하고자 하는 경우
- 해결 방안
 - **KNN(K-Nearest Neighbor) 알고리즘**





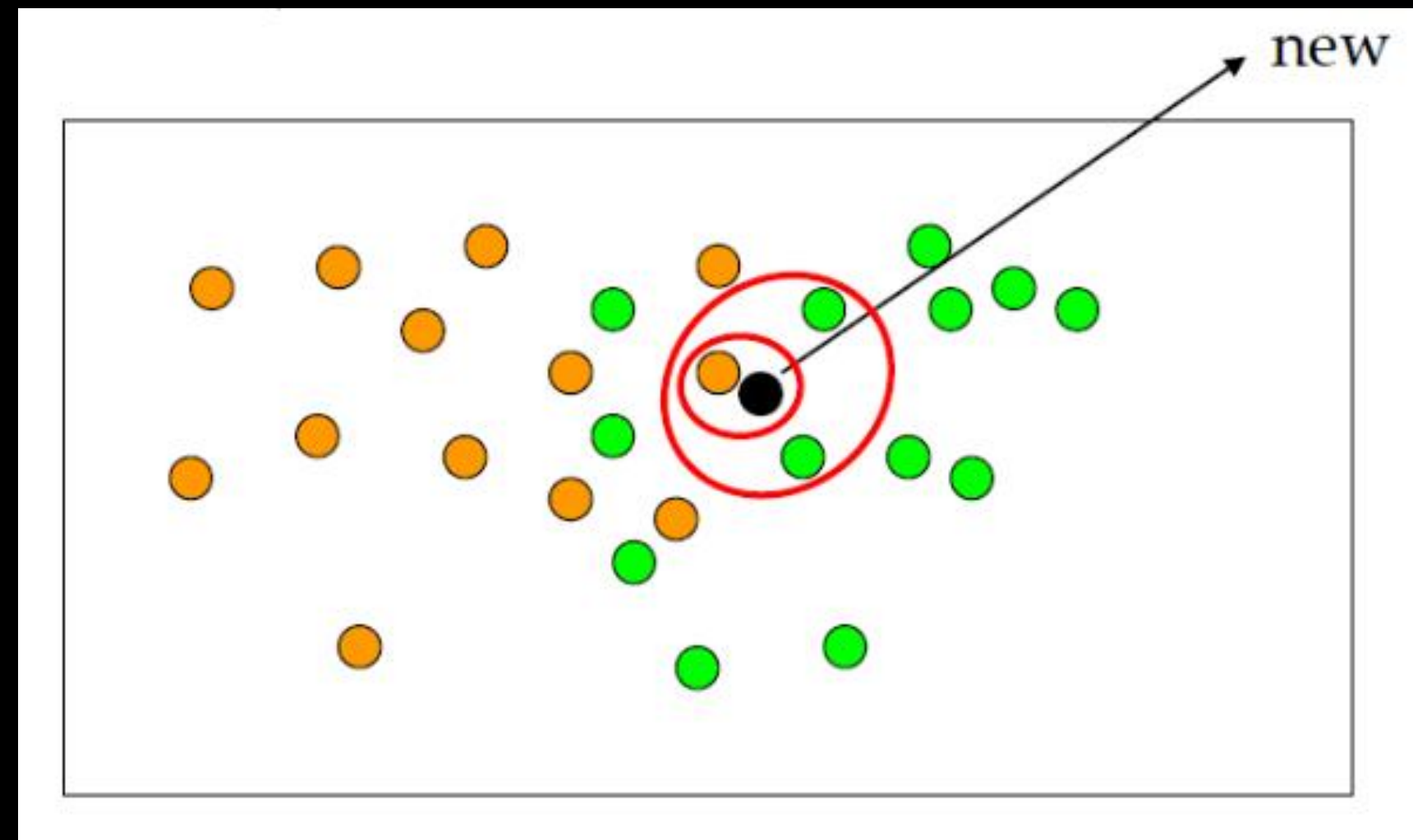
기존 데이터 가운데 가장 가까운 k개의 이웃의 정보로 새로운 데이터를 예측하는 방법론

- 유사한 특성을 가진 데이터는 유사한 범주에 속하는 경향이 있다는 가정 하에 분류
- K는 모델의 하이퍼파라미터로, 최적의 모델 성능을 찾기 위해 한 학습 사이클 종료 후 결과를 바탕으로 조정할 수 있다.





- **k값은 주로 홀수**로 설정
- 새로운 고객 데이터(검정색)이 들어왔을 때,
 - $k = 1$ 이면 주황색 클래스로 분류
 - $k = 3$ 이면 초록색 클래스로 분류





- 직관적이며 복잡하지 않은 알고리즘
- 결과 해석이 쉬움
- **K값 결정에 따라** 성능이 크게 좌우된다.
 - 이때, 가능한 K 값에 대해 모두 실험한다고 하면 학습 시간이 길어질 수 있다.



```
from sklearn.neighbors import KNeighborClassifier
```

데이터 분리

```
train_X, test_X, train_y, test_y = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

```
knn_model = KNeighborsClassifier(n_neighbors=5) 모델 생성
```

```
knn_model.fit(train_X, train_y) 모델 학습
```

```
predicted = knn_model.predict(test_X) 테스트 데이터에 대한 예측
```

- KNeighborsClassifier()
- n_neighbors = k값 설정 (되도록 홀수로 설정, default=5)

5. 분류 알고리즘 평가 지표

- 분류 모델의 성능을 평가하기 위해 사용되는 행렬

		모델이 예측한 값	
		Positive	Negative
실제 데이터	Positive	True Positive	False Negative (Type-2 error)
	Negative	False Positive (Type-1 error)	True Negative

- **Positive** (긍정 클래스, 양성): 모델이 관심 있는 클래스
- **Negative** (부정 클래스, 음성): 모델이 관심 없는 클래스
- 예시: 스팸 메일 분류기의 경우, 스팸 메일은 Positive, 비 스팸 메일은 Negative

- 분류 모델의 성능을 평가하기 위해 사용되는 행렬

		모델이 예측한 값		
		Positive	Negative	
실제 데이터	Positive	True Positive	False Negative (Type-2 error)	Recall $\frac{TP}{(TP + FN)}$
	Negative	False Positive (Type-1 error)	True Negative	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$



전체 데이터 중에서 제대로 분류된 데이터의 비율

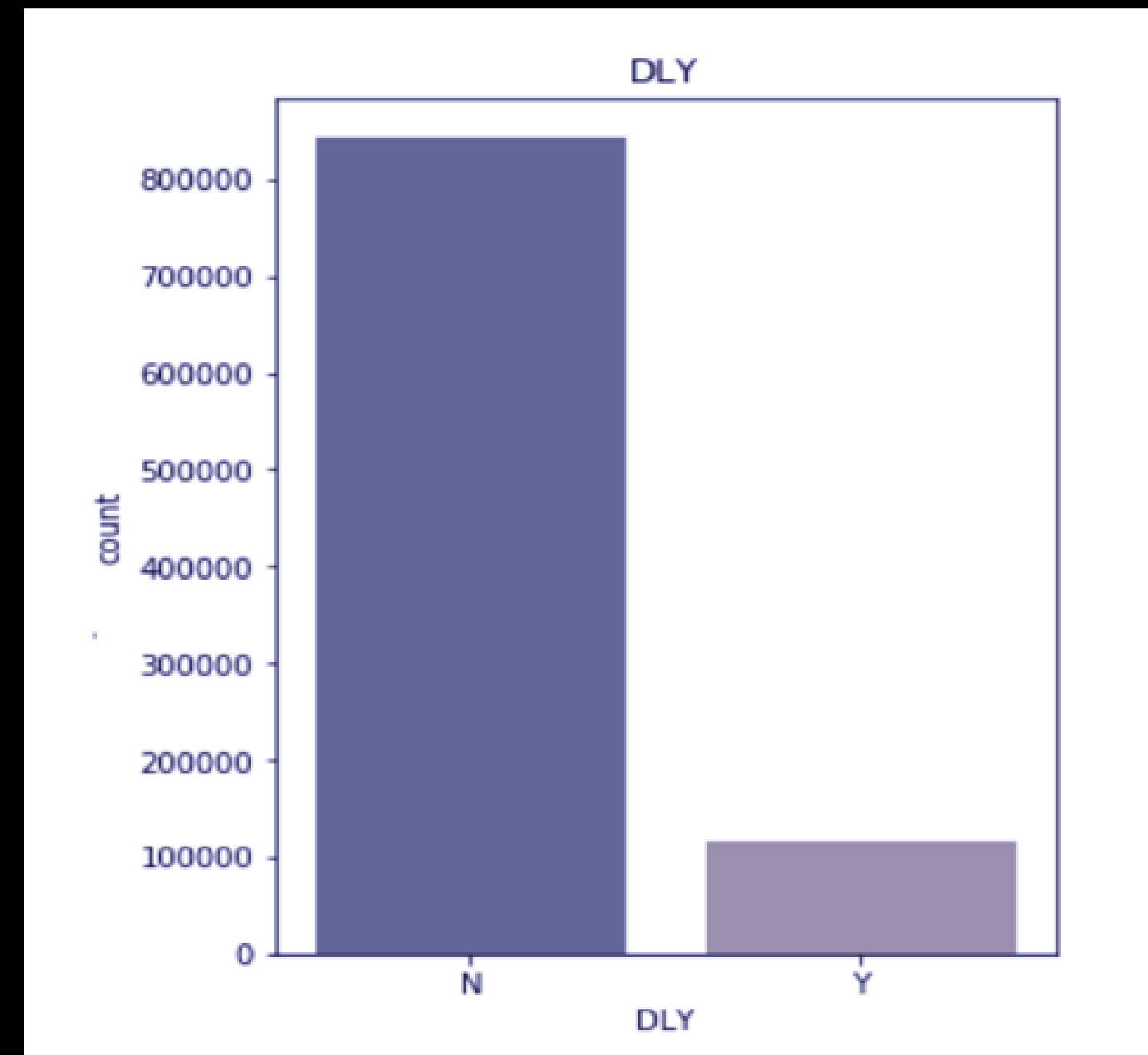
$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- 모델이 얼마나 정확하게 분류하는지를 나타냄
- 일반적으로 **분류 모델의 주요 평가 방법**으로 사용
- **클래스 비율이 불균형**할 경우, 평가 지표의 **신뢰성을 잃음**



만약 전체 100만개 항공 데이터 중

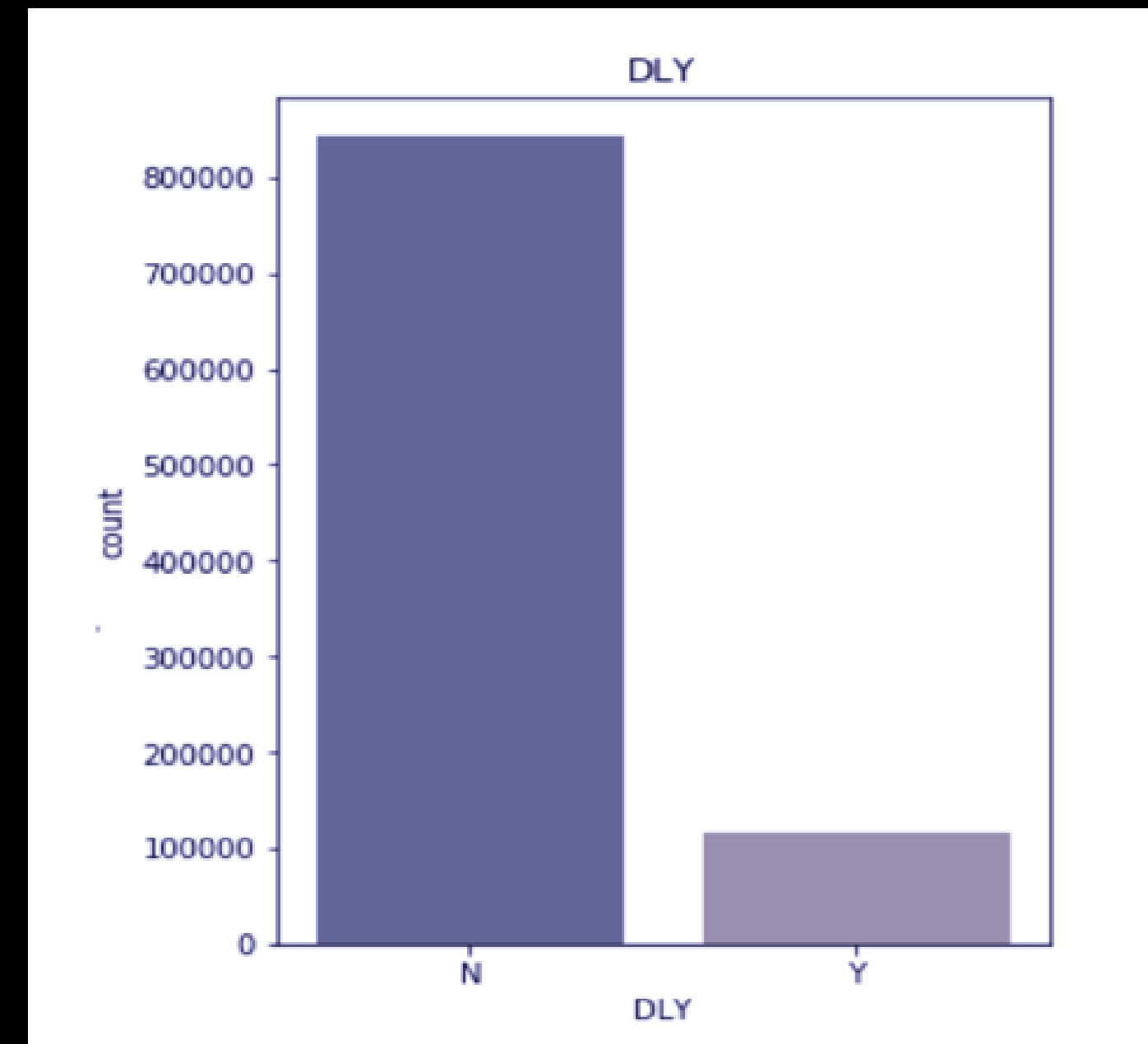
- 90만개가 정상 운행
- 10만개만 지연 운행인 데이터



$$\text{정확도} = \frac{0\text{개} + 90\text{만개}}{100\text{만개}} = 90\%$$



- 전체 결과가 모두 지연되지 않았다고 예측될 경우
- 정확도가 90% 라고 단정하기에는 위험
- 다양한 평가지표 고려 필요



$$\text{정확도} = \frac{0\text{개} + 90\text{만개}}{100\text{만개}} = 90\%$$



모델 예측이 **Positive**라고 분류한 데이터 중 **실제로 Positive**인 데이터의 비율

$$Precision = \frac{True\ Positive(TP)}{True\ Positive(TP) + False\ Positive(FP)}$$

- 모델이 예측한 긍정 사례가 실제로 맞는 비율 (=예측의 정확도)
- 일반적으로 정밀도는 **Type-1 Error**를 줄이는 것이 중요한 상황에서 주로 사용한다.
 - **Type-1 Error**: 실제로는 Negative인데 Positive로 예측한 사례



- 스팸 메일 판정 문제
 - 스팸이 아닌 정상 메일 (Negative)이 스팸(Positive) 으로 잘못 분류(False Positive) 될 경우
 - 중요한 메일을 전달 받지 못하는 상황

		모델이 예측한 값		
		Positive	Negative	
실제 데이터	Positive	True Positive	False Negative (Type-2 error)	Recall $\frac{TP}{(TP + FN)}$
	Negative	False Positive (Type-1 error)	True Negative	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$





실제로 **Positive**인 데이터 중 모델이 **Positive**로 분류한 데이터의 비율

$$Recall = \frac{True\ Positive(TP)}{True\ Positive(TP) + False\ Negative(FN)}$$

- 모델이 실제 긍정 사례를 놓치지 않는 비율 (=관심 사례 포착 정도)
- 일반적으로 재현율은 **Type-2 Error**를 줄이는 것이 중요한 상황에서 주로 사용한다.
- **Type-2 Error**: 실제로는 Positive인데 Negative로 예측한 사례



- 질병 진단
 - 암 환자(positive)를 놓치는 경우(False Negative)는 큰 위험을 초래함
 - 즉, 모델이 실제 환자를 놓치지 않도록 보장해야 함
 - Recall**을 높이면 실제 암 환자를 더 많이 포착하게 되어, 놓치는 환자를 최소화할 수 있게 됨

		모델이 예측한 값		
		Positive	Negative	
실제 데이터	Positive	True Positive	False Negative (Type-2 error)	Recall $\frac{TP}{(TP + FN)}$
	Negative	False Positive (Type-1 error)	True Negative	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$



정밀도와 재현율의 조화 평균

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

- 모델의 **정확성과 완전성을 동시에 고려**하여 모델의 성능을 평가
- **데이터 불균형이 심각할 때** 사용

실제로 **Negative**인 데이터 중 모델이 **Positive**로 분류한 데이터의 비율

$$FPR = \frac{FP}{FP + TN}$$

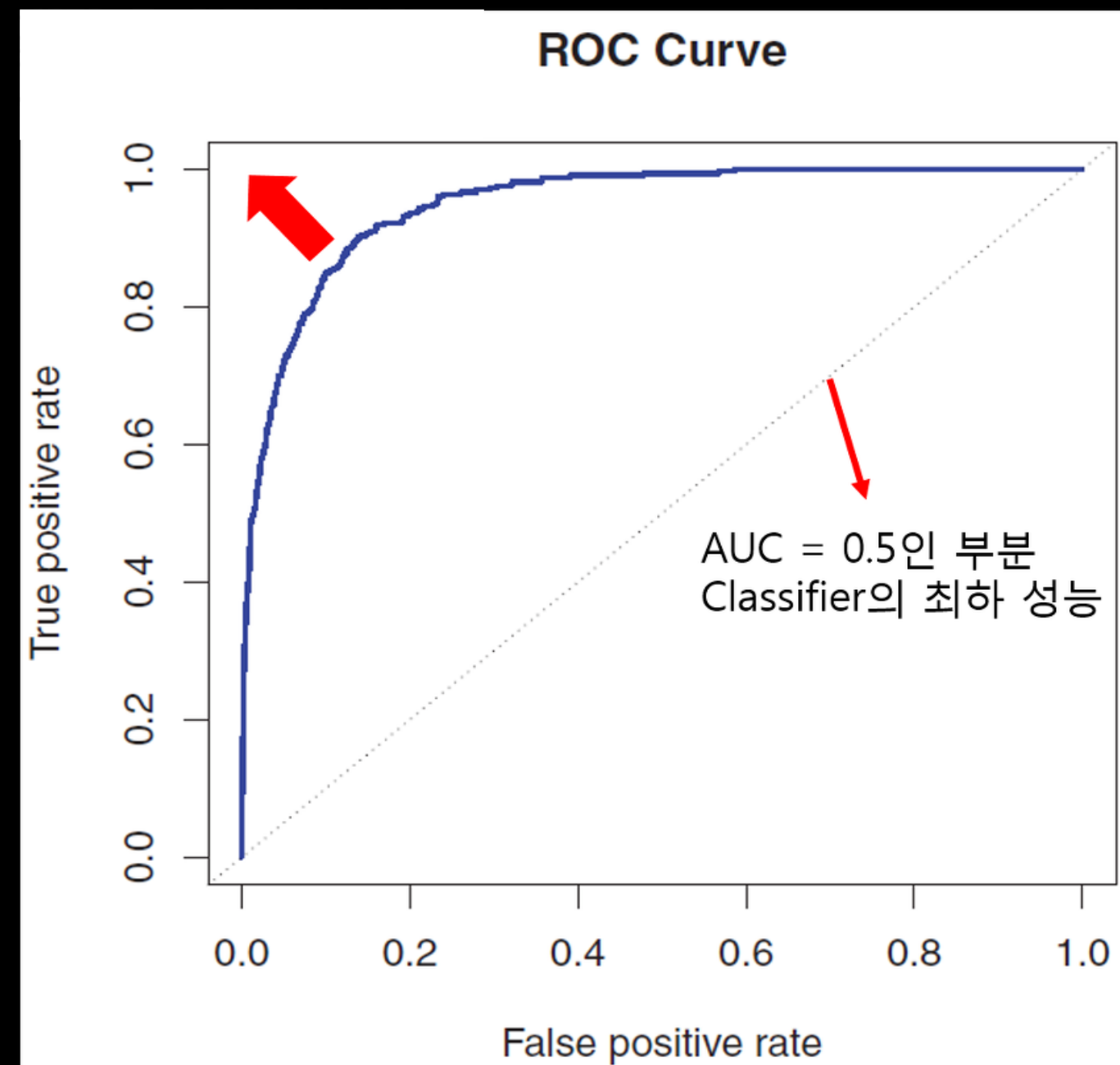
- 예시) 게임에서 비정상 사용자 검출 시,

FPR↑, 정상 사용자를 비정상 사용자로 검출하는 경우가 많음

→ 비정상 사용자에게 페널티를 부여할 경우, 선의의 사용자가 피해를 입음



- x축 False Positive Rate, y축 Recall(True Positive Rate) 값으로 시각화한 그래프
- **커브가 왼쪽 위로 당겨질 수록 Classifier의 성능이 향상**됨을 의미
- AUC(Area Under Curve) = ROC Curve 아래 면적





```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
# 검증 데이터에 대한 예측
```

```
y_pred = model.predict(X_valid)
```

평가 지표 값을 구해주는 함수 import

```
# 정확도 계산
```

```
accuracy = accuracy_score(y_valid, y_pred)
```

```
# 정밀도 계산
```

```
precision = precision_score(y_valid, y_pred)
```

```
# 재현율 계산
```

```
recall = recall_score(y_valid, y_pred)
```

```
# F1 Score 계산
```

```
f1 = f1_score(y_valid, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
print("Precision:", precision)
```

```
print("Recall:", recall)
```

```
print("F1 Score:", f1)
```